

PIXEL SOFTWARE DESIGN

Département de Recherche & Développement

Intelligence Artificielle Avancée

& Federated Learning

- Fondations de l'intelligence artificielle
- Algorithmes d'apprentissage fédéré
- Confidentialité, attaques et défenses
- Applications et grands modèles fédérés
- Framework FedX de référence
- Nouveaux fronts de recherche
- Travaux pratiques, projets et évaluation

Auteur	**Ghiloufi Jamel**
Fonction	**Architecte Logiciel Principal**
Direction	**Département de Recherche & Développement**
Service	**R&D - Intelligence Artificielle Avancée & Apprentissage Fédéré**
Niveau	**Recherche**
Édition	**Édition 2026 – 1.0**
Copyright	**© Pixel Software Design**

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Copyright et mentions légales

Éditeur	Pixel Software Design
Auteur	Ghiloufi Jamel
Fonction	Architecte Logiciel Principal
Département	Département de Recherche & Développement
Intitulé	Intelligence Artificielle Avancée & Federated Learning
Sous-titre	Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.
Édition	Édition 2026 (version 1.0)
Langue	Français
Niveau	Recherche
Collection	Supports de formation et de recherche, Pixel Software Design

Droits d'auteur

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Droits de l'auteur. Ghiloufi Jamel est l'auteur unique de l'ensemble du contenu scientifique, pédagogique et technique de cet ouvrage. Toute adaptation, traduction, reproduction ou exploitation, y compris partielle, requiert une autorisation écrite préalable.

Droits des tiers. Les algorithmes, modèles et travaux cités restent la propriété de leurs auteurs respectifs. Les références employées figurent en fin d'ouvrage, ce qui permet de remonter au travail original pour toute affirmation chiffrée.

Marques et licences externes. PyTorch, TensorFlow, Flower, Opacus, MONAI, scikit-learn et PySyft sont des logiciels libres sous licences respectives (BSD, Apache 2.0, MIT). Leur usage dans ce document est purement pédagogique et n'emporte aucune prescription commerciale.

Mentions relatives aux données et aux expérimentations

Document pédagogique à usage de formation interne. Les exemples de code fournis sont livrés à titre pédagogique ; les résultats numériques illustrés sont issus d'expérimentations conduites dans un cadre de recherche et ne constituent pas des performances de production.

Ce support synthétise les travaux de recherche du Département de Recherche & Développement de Pixel Software Design sur l'intelligence artificielle distribuée, l'apprentissage fédéré, la protection de la vie privée et les systèmes embarqués intelligents.

Sommaire

Structure détaillée du cours

Partie 1 — IA et Trois Dimensions 8

1.1 L'écosystème des modèles de génération 3D 9

1.2 La chaîne de production, de l'image au maillage 10

1.3 Les infrastructures d'inférence 11

1.4 Cinématique inverse et apprentissage 13

1.5 Estimation de pose et capture de mouvement 15

1.6 Apprentissage par renforcement et physique différentiable 16

Annexes 20

A.1 Liste des abréviations 21

A.2 Table de notation 21

Partie	Thème	Contenu	Compétence visée	Durée
Partie 1	Fondations	Statistique, optimisation, courbes d'apprentissage	Comprendre et formaliser	2-3 semaines
Partie 2	Algorithmes FL	FedAvg, convergence, hétérogénéité, système	Implémenter et régler	3-4 semaines
Partie 3	Confidentialité	DP, attaques, défenses, agrégation sécurisée	Auditer et sécuriser	2-3 semaines
Partie 4	Applications	FedLLM, santé, V2X, FedRL, TinyML, graphes	Transposer à un domaine	3-4 semaines
Partie 5	Framework FedX	Architecture, API, benchmarks, exploitation	Industrialiser	2 semaines
Partie 6	Nouveaux fronts	FedCL, FedCI, SNN, FedGL, feuille de route	Ouvrir la recherche	2 semaines
Partie 7	TP et projets	9 labs, 5 projets, corrigés, évaluation	Faire et rendre	4-6 semaines

Préface

Comment utiliser ce cours

Ce support est conçu pour un parcours de formation et de recherche couvrant l'ensemble de la chaîne de valeur de l'intelligence artificielle distribuée : les fondations mathématiques, les algorithmes d'apprentissage fédéré, les menaces et défenses, les applications de pointe, une plateforme de référence opérationnelle, et enfin les nouveaux fronts ouverts par la recherche du Département.

Ce n'est pas un manuel de référence à lire d'un trait : c'est un **manuel opératoire**. Chaque chapitre alterne exposition, formalisation, mise en œuvre numérique et évaluation. Les listings sont complets et exécutables ; les tableaux comparatifs servent de grille de décision ; les schémas d'architecture servent de support de revue de conception.

Public et prérequis

Le contenu s'adresse à des étudiants de master et de doctorat, à des ingénieurs IA et à des praticiens de la recherche appliquée. Les prérequis sont la connaissance de l'apprentissage automatique classique, de l'algorithmique numérique et des bases de probabilités. Les parties 1 à 3 sont auto-suffisantes pour un étudiant qui découvre le domaine ; les parties 4 à 7 supposent la mécanique et les outils présentés dans la partie 2.

Conventions de lecture. Une expression mathématique est encadrée par deux signes dollars ; un identifiant, un fichier ou une commande est écrit en caractères à chasse fixe ; les encadrés *Définition*, *Théorème*, *Méthode* et *Attention* signalent les points à maîtriser et les pièges fréquents. Les renvois de la forme « voir chapitre 3.2 » désignent le second chapitre de la troisième partie.

Progression pédagogique

Génération, reconstruction, cinématique et mouvement

Partie 1

IA et Trois Dimensions

La représentation tridimensionnelle est le support de la réalité virtuelle, de la simulation industrielle, de la robotique et de l'animation. Pendant des décennies, la production d'un actif trois dimensions était réservée à des dizaines de milliers d'heures de travail manuel. L'apprentissage automatique a rendu la génération accessible en quelques secondes, et a ouvert un problème nouveau : celui de l'infrastructure, du choix de représentation et de l'intégration. Ce cours décrit les outils et surtout la méthode de sélection, car la plupart des échecs d'intégration ne viennent pas du modèle mais du mauvais choix de point d'entrée.

Objectifs de la partie

- Comparer les principaux modèles de génération 3D sur le critère qui compte : la contrainte la plus dure de votre usage.
- Décrire la chaîne complète, de l'image d'entrée à l'export de maillage exploitable, et nommer l'étape qui coûte le plus cher.
- Choisir entre un agrégateur de modèles, une inférence par interface, un service empaqueté et une orchestration locale.
- Expliquer pourquoi un réseau de neurones accélère la cinématique inverse sans la remplacer.
- Distinguer capture monoculaire, stéréoscopique et volumétrique, et leurs contraintes de contrat.
- Expliquer ce qu'un simulateur d'apprentissage par renforcement peut et ne peut pas transférer au monde réel.

Plan de la partie

1. L'écosystème des modèles de génération 3D
2. La chaîne de production, de l'image au maillage
3. Les infrastructures d'inférence
4. Cinématique inverse et apprentissage
5. Estimation de pose et capture de mouvement
6. Apprentissage par renforcement et physique différentiable

Chapitre 1.1

L'écosystème des modèles de génération 3D

Les modèles se distinguent moins par la qualité de leurs images de synthèse que par le point où ils échouent.

Une carte graphique sérieuse exécute désormais la reconstruction d'un objet en quelques secondes. La différence entre les modèles disponibles n'est donc plus une question de faisabilité, mais de domaine de validité : chacun réussit excellemment dans une zone et échoue nettement à sa frontière, et c'est toujours cette frontière qui décide de l'utilité réelle.

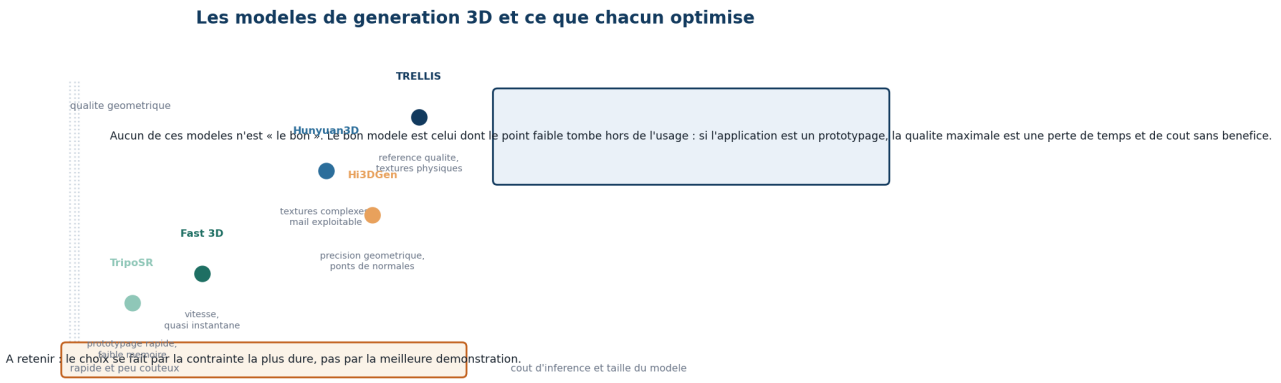


Figure 1 – Les modèles de génération 3D et ce que chacun optimise

Famille	Principe	Atout	Limite principale
Diffusion de vue	génère des vues multiples cohérentesqualité visuelle élevée	lent, mémoire importante	
Champ de densité	prédit un volume de probabilité de surface	très rapide, topologie correcte	tops trouées, textures floues
Surface implicite	extrait l'isosurface d'un champ scalaire	maillage étanche, topologie fiable	extraction coûteuse
Graphe de scène	prédit directement un graphe de faces	maillage natif, pas de conversion	mémoire très élevée, résolution plafonnée
Champ de radiance	rend une scène par lancer de rayons	rendu photoréaliste excellent	ne produit pas de géométrie exploitable

Tableau 1 – Les familles de modèles et leurs compromis

La dernière ligne est la plus coûteuse à découvrir tard. Un système de rendu à lancer de rayons produit des images remarquables, et ne fournit aucun actif utilisable dans un moteur de jeu ou une chaîne de fabrication.

Choisir un modèle en cinq questions, dans cet ordre

- **Quel est l'usage en aval ?** Affichage, impression, animation, mesure, ou simulation. L'usage détermine la représentation exigée, donc tout le reste.

2. **Quelle est la contrainte la plus dure ?** Latence, mémoire vidéo, précision géométrique, ou fidélité des matières. On choisit sur la contrainte, pas sur la démonstration.
3. **Quelle diversité d'objets ?** Un modèle excellent sur un domaine étroit et médiocre ailleurs impose de connaître son domaine avant de l'adopter.

4. **Quel est le coût d'une erreur ?** Une erreur visuelle se corrigé à la retouche ; une erreur géométrique se propage jusqu'à la pièce fabriquée.
5. **Les sorties sont-elles exploitables ?** Format, topologie, carte de texture, système de coordonnées. Un modèle dont la sortie exige un post-traitement lourd a un coût caché souvent supérieur à son coût d'exécution.

La qualité d'image n'est pas la qualité de l'actif

Un modèle peut produire un rendu<Integer> superbe et un maillage inutilisable : faces non orientées, normales incohérentes, trous dans les zones occultées, coordonnées de texture absentes. L'évaluation d'un modèle de reconstruction doit donc porter sur l'actif exporté et non sur l'image produite. Les métriques habituelles de comparaison visuelle ne mesurent ni les erreurs d'orientation des faces, ni les défauts de couture, ni la topologie.

Astuce

Le seul test qui vaille est de prendre dix objets de votre domaine réel, les passer dans le modèle, et mesurer le temps de correction manuelle. Ce chiffre décidera du choix mieux que toute comparaison publiée.

A retenir

- Les modèles 3D se distinguent par leur domaine de validité, pas par leur qualité moyenne.
- Le rendu spectacular et l'actif exploitable sont deux propriétés distinctes, et la seconde décide de l'utilité.
- L'évaluation porte sur le maillage exporté, jamais sur l'image de synthèse.
- Le critère de sélection est la contrainte la plus dure de votre usage.

Chapitre 1.2

La chaîne de production, de l'image au maillage

L'inférence n'est qu'une étape. Les deux autres étapes coûtent souvent plus cher qu'elle.

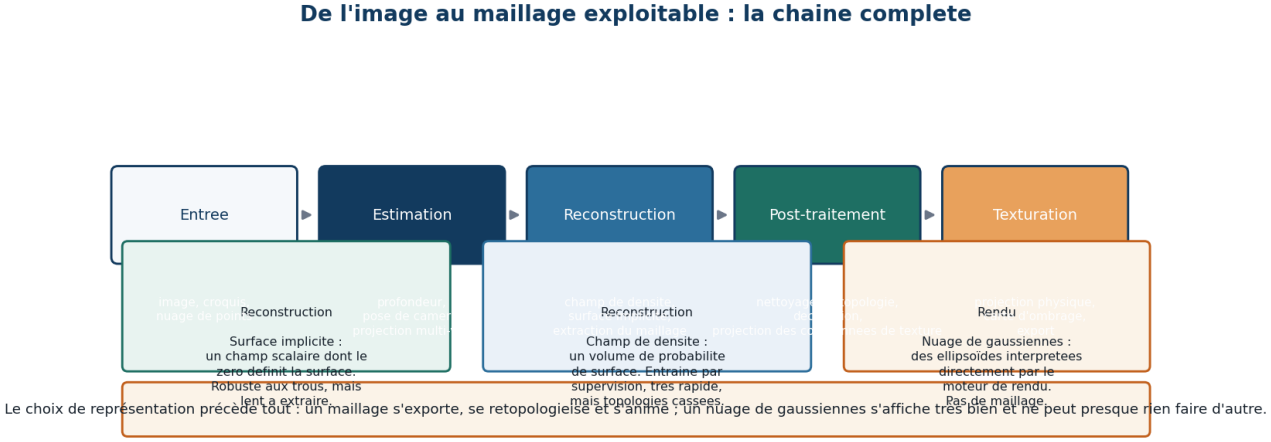


Figure 1 – De l'image au maillage exploitable : la chaîne complète

Une chaîne de reconstruction comporte toujours les mêmes étapes, que le modèle en porte le nom ou non : acquisition, estimation de la géométrie, reconstruction, post-traitement, et habillage. Chacune peut échouer silencieusement, et les erreurs se propagent d'étape en étape : une profondeur estimée faux produit un maillage mal orienté, qu'aucun post-traitement ne rattrapera.

$$M = \leq ft(V, F, N, U, Mat), Mat \in \leq ft \text{ diffuse, rugueuse, métallique, transmission } ^{n\square 3\square} \quad (M.1)$$

Un actif exploitable est un tuple : sommets, faces, normales, coordonnées de texture et paramètres de matériau. Les modèles récents fournissent les trois premiers ; les deux derniers restent très souvent à produire, et c'est précisément ce travail manuel qui rend le résultat présentable.

- **Nettoyage de topologie :** suppression des faces isolées, suppression des composantes non connectées, et consolidation des sommets dupliqués. Rapide, et souvent négligé.
- **Décimation :** réduction du nombre de faces en préservant la silhouette. C'est l'opération qui décide du budget réel, car c'est elle qui détermine la fluidité à l'affichage.
- **Projection des coordonnées de texture :** choix de la paramétrisation. Les méthodes par développement et par projection sphérique ont des coutures ; les méthodes basées sur le diagramme de Beatrice sont plusrobustes mais plus lentes.
- **Habillage :** projection de la texture générée sur le maillage précédent, puis estimation de paramètres de matériau pour rendre correctement sous un éclairage physique.
- **Validation géométrique :** auto-intersection, orientation des normales, manifoldness, épaisseur minimale. C'est cette étape qui sépare un actif affichable d'un actif dommageable dans une chaîne de fabrication.

Contrôle qualité d'un actif reconstruit, en sept points

- **Manifoldness** : chaque arête doit appartenir à exactement deux faces.

2. **Orientation** : les normales doivent pointer vers l'extérieur, sans exception ni incohérence.
3. **Auto-intersection** : le maillage ne doit pas se traverser lui-même à l'échelle d'usage.
4. **Trous et composantes** : aucune face isolée, aucune composante détachée non intentionnelle.
5. **Coutures** : la paramétrisation ne doit pas produire de rupture visible dans les zones vues.
6. **Densité** : le nombre de faces doit correspondre au budget d'affichage cible, pas au maximum possible.
7. **Échelle et repère** : dimensions réelles, origine au bon endroit, axes cohérents avec le moteur en aval.

Les quatre premiers points peuvent être vérifiés automatiquement et doivent l'être : ils ne dépendent pas de l'œil et leur oubli se paie très tard.

Un maillage dont les normales ne sont pas cohérentes peut s'afficher correctement dans un moteur qui recalcule les normales, et produire des ombres fausses dans un moteur qui ne le fait pas. Le symptôme apparaît alors dans un autre outil, sur une autre machine, chez un autre prestataire. Vérifier et corriger l'orientation une seule fois, à la production, coûte une heure ; le retrouver en production coûte un cycle complet d'intégration.

Le maillage orientations incohérentes

A retenir

- La chaîne comporte cinq étapes, et le modèle n'en porte qu'une seule.
- Les erreurs de profondeur se propagent jusqu'à la texture et ne sont rattrapables par aucun post-traitement.
- Un actif exploitable inclut normales et matériaux, pas seulement sommets et faces.
- La validation géométrique doit être automatisée, car ses premiers points ne dépendent pas de l'œil.

Chapitre 1.3

Les infrastructures d'inférence

La question d'infrastructure se tranche sur une seule donnée : la donnée a-t-elle le droit de sortir ?

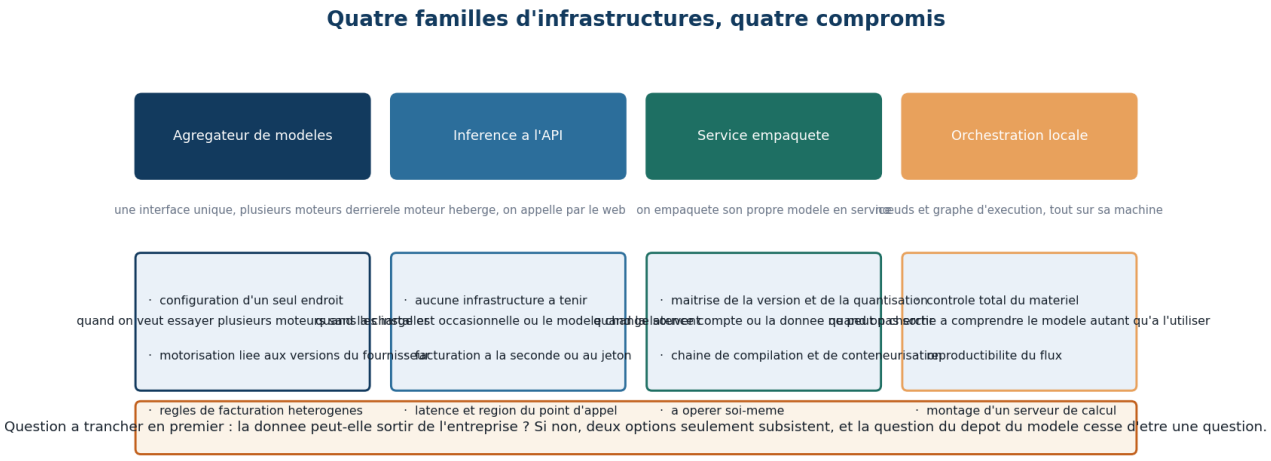


Figure 1 – Quatre familles d'infrastructures, quatre compromis

Famille	Ce que vous gérez	Facturation	LatenceContrôle de la version	
Agrégateur de modèles	rienpar génération, souvent au jeton	moyenne à élevée	imposé par le fournisseur	
Inférence par interface	rien	à la seconde de calcul, ou par volume	variable selon la charge	figée à l'appel
Service empaqueté	la conteneurisation et la compilation	à votre infrastructure	maîtrisée	maîtrisé
Orchestration locale	le matériel et le graphe d'exécution	à votre infrastructure	maîtrisée	maîtrisé

Tableau 1 – Les quatre familles en détail

Le contrôle de la version est la colonne la plus négligée et la plus structurante. Un fournisseur d'inférence peut remplacer un modèle entre deux appels : votre résultat devient non reproductible sans que rien n'ait changé dans votre code.

Trancher l'infrastructure en quatre questions

- **La donnée peut-elle quitter l'entreprise ?** Si non, deux options subsistent : service empaqueté ou orchestration locale. Cette question se pose avant toute autre.

2. **Quelle part de la charge est expérimentale ?** Si la majorité des requêtes sont des essais, l'inférence par interface évite tout investissement et se remplace sans/kubernetes.
3. **Quelle latence faut-il garantir ?** Une latence maximale engage à un service maîtrisé, car un fournisseur distant n'offre aucun engagement contractuel sur le centime de milliseconde.
4. **Faut-il pouvoir reproduire un résultat dans six mois ?** Si oui, le modèle doit être archivé localement avec son empreinte, quelle que soit l'inférence choisie.

Un usage réel combine les deux : interface pour l'exploration, service maîtrisé pour la production, et toujours une copie locale du modèle archivé.

Le coût caché n'est pas l'inférence

Sur une génération de quelques secondes, le coût d'inférence est négligeable devant trois autres postes : la file d'attente avant la première image, la correction manuelle des actifs produits, et le stockage des versions intermédiaires. Une équipe qui compare deux fournisseurs sur le prix à la génération optimise le poste le moins coûteux de la chaîne. Le bon indicateur de comparaison est le coût total par actif livré et validé.

Astuce

Archivez toujours le modèle avec son empreinte et sa version de format. Un résultat obtenu avec une référence de modèle mouvante n'est pas reproductible, quelle que soit la qualité de l'infrastructure qui l'a produit. Ce conseil vaut pour tout système d'inférence, et il est systématiquement ignoré.

A retenir

- La contrainte qui décide de tout est le droit de sortie de la donnée.
- Le contrôle de la version du modèle détermine la reproductibilité, et se négocie rarement.
- Comparer deux fournisseurs sur le prix à la génération optimise le poste le moins coûteux.
- Le bon indicateur est le coût total par actif livré et validé.

Chapitre 1.4

Cinématique inverse et apprentissage

L'apprentissage n'accélère pas la cinématique inverse : il en apprend l'initialisation.

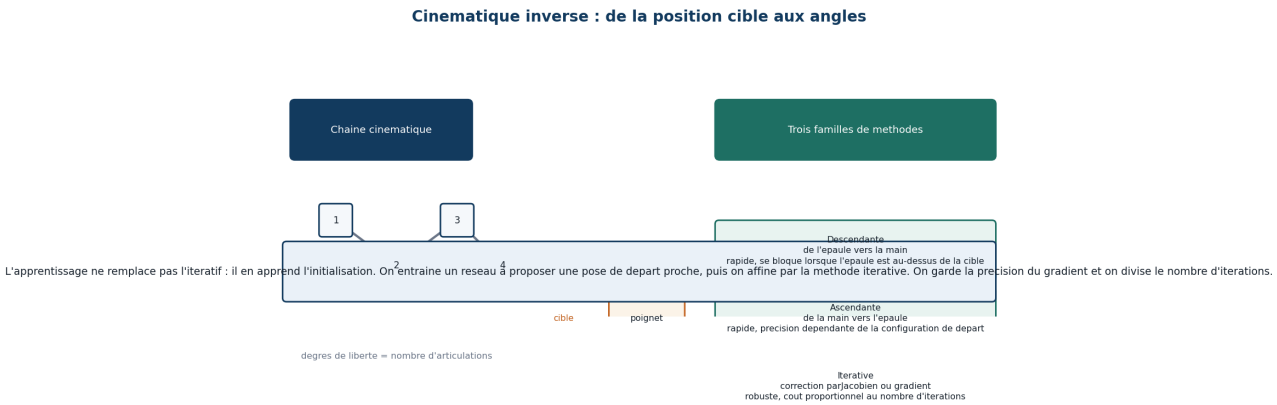


Figure 1 – Cinématique inverse : de la position cible aux angles

$$\min_{\boldsymbol{\theta}} \left\| \mathbf{f}(\boldsymbol{\theta}) - \mathbf{p}^{*} \right\|^{2} + \lambda \left\| \boldsymbol{\theta} - \boldsymbol{\theta}_{0} \right\|^{2} \tag{M.2}$$

Deux termes, deux rôles : le premier atteint la cible, le second empêche une solution aberrante. Sans le terme de régularisation, une chaîne à six degrés de liberté atteint n'importe quelle position de nombreuses manières, dont certaineserinversent le bras. Le choix du pondérateur est donc un choix de posture, pas un réglage numérique.

Famille	Sens de propagation	Coût	Échec typique
Descendante	de l'épaule vers la main	constant, faibleposition d'épaule hors de l'espace atteignable	
Ascendante	de la main vers l'épaule	constant, faible	précision dépendante de la configuration de départ
Descendante itérative	correction par Jacobien	proportionnel aux itérations	mauvaise configuration initiale, singularité
Descente de gradient	correction par dérivée	proportionnel aux itérations	optimum local
Apprentissage	proposition d'une initialisation	quasi nul	hors de la distribution d'entraînement

Tableau 1 – Les trois familles de résolution et leurs propriétés

La dernière ligne est la seule qui soit nouvelle. Le réseau ne résout rien : il fournit un point de départ si bon que deux ou trois itérations suffisent, contre vingt ou trente.

```python
import numpy as np


def cinematique_descendante(pointeur_cible: np.ndarray, longueurs: list[float],
angle_initial: float = 0.0) -> np.ndarray:
"""Propagation descendante, en dynamique inverse ou geometrique.

On fixe l'angle de l'epaule, puis on enchaîne les positions. Si la
cible n'est pas atteinte, on ajuste l'angle d'epaule par un balayage
grossier puis par affinage : c'est ce second passage qui transforme
une methode qui "ne converge pas toujours" en methode qui converge.
"""
nb = len(longueurs)
angles = np.full(nb, angle_initial)

def evaluer(theta: np.ndarray) -> float:
position = np.zeros(2)
direction = theta[0]
for i in range(nb - 1):
position = position + longueurs[i] * np.array(
[np.cos(direction), np.sin(direction)])
direction += theta[i + 1]
position = position + longueurs[-1] * np.array(
[np.cos(direction), np.sin(direction)])
return float(np.linalg.norm(position - pointeur_cible))

# balayage grossier, puis affinage par dichotomie sur chaque angle
grille = np.linspace(-np.pi, np.pi, 25)
for _ in range(3):
for i in range(nb):
meilleur, meilleur_cout = angles[i], evaluer(angles)
for candidat in grille:
essai = angles.copy()
essai[i] = candidat
cout = evaluer(essai)
if cout < meilleur_cout:
meilleur, meilleur_cout = candidat, cout
angles[i] = meilleur
return angles


def resoudre_reseau(proposeur, cible: np.ndarray, affiner: int = 3) -> np.ndarray:
"""Apprentissage propose, methode iterative affine.

Le reseau fournit une initialisation en une passe. On garde la
methode iterative pour l'affinage, parce qu'elle garantit l'optimalite
locale : le reseau garantit la rapidite, pas l'exactitude.
"""
theta = proposeur(cible)
for _ in range(affiner):
ecart = cible - forward(theta)
jacobienne = forward_jacobienne(theta)
theta = theta + np.linalg.solve(jacobienne.T @ jacobienne,
jacobienne.T @ ecart)
return theta
```

Listing 1 — Résolution de cinématique inverse : géométrique et apprentissage

Entraîner un proposeur de pose : les quatre règles

- **Générer les poses par tirage aléatoire uniforme sur l'espace des articulations**, et non en simulant un mouvement réaliste. Le modèle doit couvrir tout l'espace atteignable, y compris les postures qu'aucun être humain ne prendrait pas.

2. **Filtrer par accessibilité** : ne garder que les poses dont l'atteinte de la cible est possible avec la longueur du bras réelle, sous la contrainte d'évitement d'obstacle.

3. **Entraîner par régression sur la pose de la cible, pas sur les angles**. La cible est l'entrée du service ; les angles sont une paramétrisation interne. Entraîner sur les angles apprend au modèle un repère arbitraire.
4. **Ajouter une composante de gestion des ambiguïtés.** Quand deux postures atteignent la même cible, la loss doit les considérer toutes deux correctes, sinon le modèle choisit arbitrairement et bascule d'une solution à l'autre d'une image à l'autre.

Un réseau qui prédit directement les angles articulaires sans aucune vérification donne des poses qu'il n'a jamais apprises, et rien ne le signale. La parade est simple et jamais optionnelle : conserver la résolution itérative en raffinement, et ajouter une vérification explicite que la position atteinte est bien celle demandée, à une tolérance près. Un modèle rapide mais non vérifié est une source de défauts intermittents, c'est-à-dire la pire catégorie de défaut possible.

L'abandon de l'itératif au profit du réseau

A retenir

- L'apprentissage ne remplace pas la résolution itérative : il en fournit l'initialisation.
- Le terme de régularisation dans la fonction de coût choisit une posture, pas un réglage.
- Entraîner sur les angles apprend au modèle un repère arbitraire : il faut entraîner sur la cible.
- Une résolution non vérifiée produit des défauts intermittents, la plus mauvaise catégorie possible.

Chapitre 1.5

Estimation de pose et capture de mouvement

La difficulté de la capture de mouvement n'est pas de voir un squelette : c'est d'en déduire une position absolue.

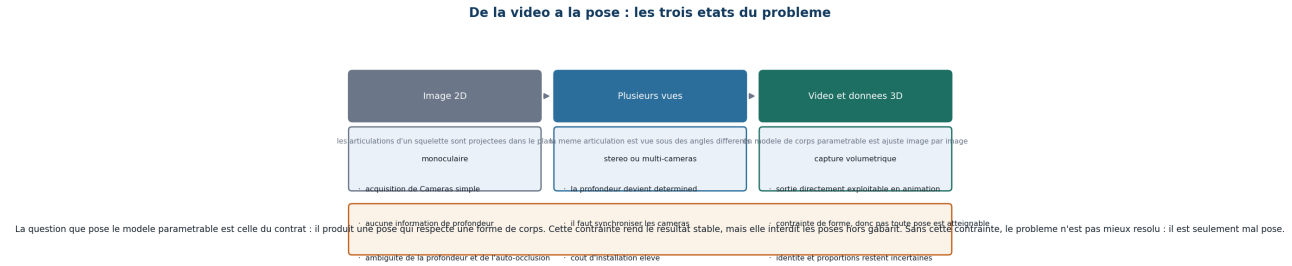


Figure 1 – De la vidéo à la pose : les trois états du problème

Un réseau de détection de squelette sur image est devenu un problème résolu : la question de savoir où se trouve un coude dans une image n'a plus d'intérêt. Le problème ouvert commence après : une image ne contient qu'une projection, et une projection ne détermine pas une position. C'est cette ambiguïté, et non la détection, qui sépare un système de démonstration d'un système exploitable.

$$\pi : \mathbb{R}^3 \to \mathbb{R}^2,\ \exists\ (x_1, x_2) \neq (x_1', x_2') : \pi(x_1) = \pi(x_2')$$

$$\text{hatboldsymbol}\beta = \operatorname{argmin}_{\text{boldsymbol}\beta} \leq\text{ft}| \pi\ (\ T,\ \text{boldsymbol}\beta\ ) - \text{mathbf}J_{2D}\ |^2_W + \lambda \leq\text{ft}|\ \text{boldsymbol}\beta - \text{boldsymbol}\beta_{prev}\ |^2_W \qquad \text{(M.3)}$$

La première expression dit l'essentiel : la projection n'est pas injective, donc la position n'est pas déterminée. La seconde est la résolution du problème par un modèle de corps paramétré, dont la contrainte de forme transforme un problème mal posé en problème bien posé – au prix de restreindre l'ensemble des poses possibles.

	Monoculaire	Stéréoscopique	Volumétrique
Entrées	une ou plusieurs caméras 2Ddeux caméras calibrées	plusieurs vues calibrées	
Profondeur	déterminée par le modèle de corps	déterminée par la géométrie	déterminée par la triangulation
Sortie	positions 3D absolues	positions 3D absolues	positions 3D absolues
Coût matériel	très faible	moyen, avec étalonnage	élevé, plusieurs caméras
Problème dominant	auto-occlusion,hors de champ	désynchronisation	faisabilité de l'installation
Usage typique	animation, ergonomie, guidage	capture de studio	analyse clinique, recherche

Tableau 1 – Trois états du problème, trois contrats

L'étalonnage est la pierre angulaire de la deuxième et de la troisième colonne, et il est presque toujours sous-estimé : une caméra déplacée de deux centimètres introduit une erreur de profondeur qui invalide la reconstruction.

Préparer une installation de capture multi-vues

- **Fixer le repère et le mesurer**, pas l'estimer. Un repère posé à vue d'œil rendra toutes les mesures ultérieures interprétables de travers.

2. **Étalonner chaque paire de caméras séparément**, avec une cible plane et une méthode d'estimation documentée. Enregistrer les paramètres : ils seront nécessaires à tout recalcul.
3. **Vérifier la précision résiduelle** sur une position connue, et la mesurer. Une incertitude de deux centimètres invalide toute mesure articulaire fine.
4. **Synchroniser les flux.** Une gigue temporelle de dix millisecondes entre deux caméras produit un bruit articulaire d'un centimètre : l'origine n'est pas dans le modèle.
5. **Prévoir la maintenance**, car une caméra déplacée reste silencieusement fausse jusqu'au réétalonnage.

L'étape 5 est celle qu'on oublié, et la seule dont l'omission produise une corruption lente et progressive des résultats.

Astuce

Pour un guidage de robot par imitation, l'étape de lissage temporel vaut autant que la qualité du modèle de détection. Un filtre qui réduit la gigue de mesure dégrade aussi la réactivité : le compromis doit être réglé sur la vitesse de mouvement réelle, pas choisi par défaut.

A retenir

- La détection d'un squelette sur image est résolue ; la reconstruction de la position ne l'est pas.
- La projection d'un point trois dimensions sur une image n'est pas injective, donc la position est ambiguë.
- Un modèle de corps paramétré rend le problème bien posé au prix d'une restriction des poses.
- L'étalonnage, pas le modèle, est la source dominante d'erreur en capture multi-vues.

Chapitre 1.6

Apprentissage par renforcement et physique différentiable

Le simulateur apprend, mais il apprend dans un monde qui n'est pas le nôtre. Tout l'art est dans l'écart.

Faire apprendre à un robot à marcher par essai-erreur est impossible en conditions réelles : les chutes détruisent le robot, et le nombre d'essais requis dépasse largement ce que l'on peut solder. L'apprentissage par renforcement déplace donc l'expérimentation dans un simulateur, où des millions de trajectoires sont produites sans coût. Le problème n'est pas résolu : il est déplacé, et le déplacement crée un écart qu'il faut mesurer.

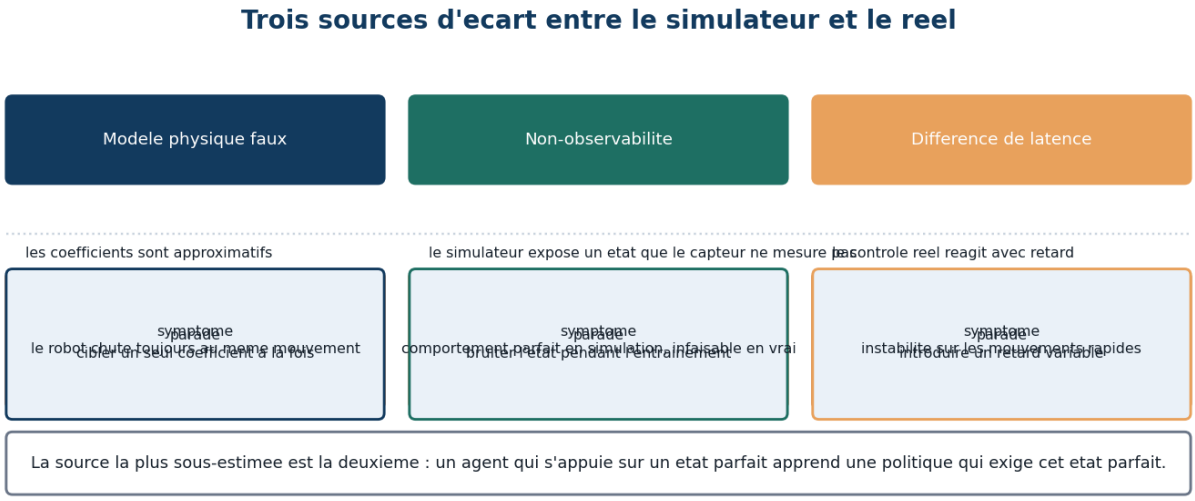


Figure 1 – Trois sources d'écart entre le simulateur et le réel

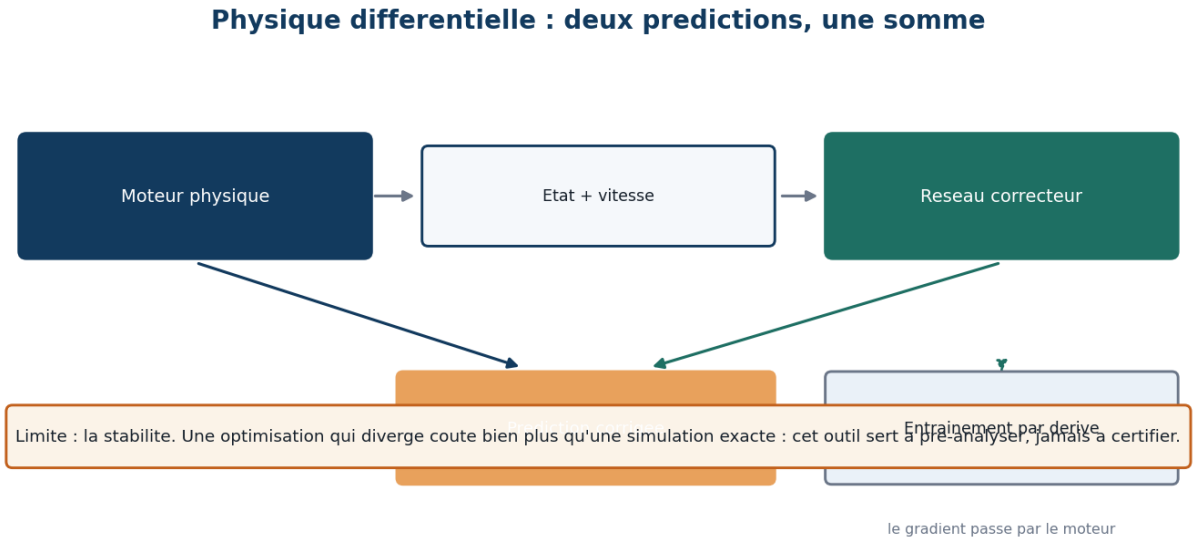


Figure 2 – Physique différentiable : deux prédictions, une somme

$$\boldsymbol{\theta}^{*} = \arg\max_{\boldsymbol{\theta}} \mathbb{E}\left[\sum_{t=0}^{T} \gamma^{t} r(s_t, a_t) \right], \text{écart } \Delta(s) = \left| s_{\square 1 \square} - s_{\square 0 \square} \right| \quad \text{(M.4)}$$

Les deux expressions doivent être lues ensemble. La première produit une politique optimale dans le simulateur. La seconde mesure ce qui manque pour qu'elle soit optimale ailleurs. Toute la difficulté du domaine tient dans l'estimation de cette différence, et aucune des deux ne se dispensé de l'autre.

Source	Mécanisme	Symptôme observé	Parade
Modèle physique faux	les coefficients sont approximatifsle robot chute systématiquement au même mouvement	identifier le coefficient fautif par essai ciblé	
Non-observabilité	le simulateur expose des états que le capteur ne mesure pas	comportement correct en simulation, impossible à réaliser	bruiter l'état pour forcer la robustesse à l'incertitude
Différence de latence	le contrôle réel réagit avec retard	oscillation ou instabilité sur les mouvements rapides	introduire un retard variable dans le simulateur

Tableau 1 – Trois sources d'écart entre le simulateur et le réel

La deuxième source est la plus sous-estimée. Un agent qui s'appuie sur un état parfait apprend une politique qui exige cet état parfait, et qui s'effondre dès qu'il est partiellement bruité.

> **Le simulateur qui ment avec précision**
> Un simulateur précise mais faux est plus dangereux qu'un simulateur approximatif mais honnête, parce qu'il produit des résultats qui semblent fiables. La parade n'est pas d'augmenter la fidélité du modèle physique, mais de rendre l'écart explicite : mesurer l'erreur de prédiction du simulateur sur des trajectoires réelles, et la publier à côté des gains obtenus. Un simulateur dont l'erreur mesurée est stable et comprise devient un instrument ; un simulateur dont l'erreur est inconnue reste une hypothèse.

Physique différentiable : rendre les lois apprenables

- **Principe :** le moteur de simulation est instrumenté pour dériver, et un réseau de neurones corrigé en sortie la prévision physique. Le moteur fournit la structure et la stabilité, le réseau apprend la part qui échappe au modèle.
- **Gain attendu :** des simulations de systèmes autrement trop coûteuses — tissus, fluides, contacts multiples — deviennent interactives, donc utilisables pour de l'optimisation.
- **Limite :** la stabilité. Un moteur différentiable peut être X peu fiable à laanj, et une optimisation qui diverge coûte bien plus qu'une simulation exacte.
- **Usage correct :** pré-analyse, criblage de scénarios, apprentissage de contrôles. Usage incorrect : validation d'un résultat critique, où seule la physique non modifiée fait foi.

Travaux pratiques 1.1 – Transférer une politique apprise vers un cadre incertain

Duree : 3 h | Niveau : Avance | Outils : Un simulateur de contrôle plan à deux degrés de liberté, et une politique entraîné par apprentissage par renforcement

Entraîner une politique de contrôle simple dans un simulateur, puis mesurer l'effet du bruit d'observation sur ses performances.

Deroulement

1. Entraîner la politique dans le simulateur sans bruit, et relever le coût final atteint.
2. Réentraîner la même politique avec un bruit gaussien sur l'observation, et relever le coût final.
3. Comparer les deux courbes d'apprentissage, et identifier la phase pendant laquelle le bruit change le plus la dynamique.
4. Mesurer la dégradation des performances en fonction de l'écart-type du bruit, sur cinq valeurs croissantes.
5. Écrire la conclusion en trois lignes : quelle source d'écart du tableau previous est illustrée, et si le bruit d'état ou le bruit d'action la reproduit le mieux.

Bareme indicatif

Critere	Points
Les deux entraînements sont menés avec la même graine et le même budget	3
La comparaison est chiffrée et non seulement qualitative	3
La source d'écart identifiée est justifiée par l'observation	4

A retenir

- L'apprentissage par reinforcement déplace l'expérimentation, il ne supprime pas le problème de l'échec.
- Trois sources d'écart séparent le simulateur du réel, et la non-observabilité est la plus sous-estimée.
- Bruiter l'état d'observation pendant l'entraînement est le moyen le plus efficace de fiabiliser le transfert.
- La physique différentiable sert à pré-analyser, pas à certifier.

Abréviations, notation et ressources de consultation

Annexes

Annexes

Ce cours se termine par deux annexes de référence. La première donne la signification de chaque sigle employé, sous les deux formes — sigle et développé — car la litterature emploie indifféremment l'une ou l'autre. La seconde donne la signification de chaque symbole mathématique : elle permet de relire une démonstration sans revenir au chapitre dont elle est extraite, et de vérifier qu'un même symbole garde le même sens d'un chapitre à l'autre.

Chapitre A.1

Liste des abréviations

Les sigles ci-dessous sont ceux que le cours emploie. Chacun est donné avec sa forme développée et le sens précis qu'il prend ici, qui n'est pas toujours celui qu'on lui donne ailleurs.

Un sigle employé sans être développé oblige le lecteur à le reconnaître au passage, ou à l'abandonner. Le tableau ci-dessous ferme la liste de ceux que le cours emploie : il est exhaustif pour ce document, et non pour la discipline.

Sigle	Forme développée	Sens dans ce cours
CG	Computer Graphics	Infographie, c'est-à-dire synthèse d'images numériques.
CUDA	Compute Unified Device Architecture	Modèle de programmation des accélérateurs graphiques.
GLTF	GL Transmission Format	Format d'échange d'objets tridimensionnels.
LiDAR	Light Detection and Ranging	Mesure de distance par impulsion laser.
NeRF	Neural Radiance Field	Représentation neuronale d'une scène, par radiance et densité.
OBJ	Wavefront Object	Format de fichier d'objets tridimensionnels, sous forme texte.
PBR	Physically Based Rendering	Rendu fondé sur le principe physique de conservation de l'énergie.
PLY	Polygon File Format	Format de maillage tridimensionnel.
RGB-D	Red Green Blue and Depth	Acquisition d'une couleur et d'une profondeur synchronisées.
SLAM	Simultaneous Localization and Mapping	Localisation et cartographie simultanées d'un robot.

Tableau 1 – Abréviations employées dans le cours

Chapitre A.2

Table de notation

Les symboles ci-dessous gardent le même sens d'un chapitre à l'autre. Lorsqu'un chapitre doit réemployer un symbole pour un autre objet, il le redéclare sur la page même.

La notation est celle du cours : vecteurs en gras, matrices en majuscules grasses, scalaires en italique. Les espérances sont notées $\mathbb{E}$, les lois de données $\mathcal{D}$, les espaces de paramètres Θ.

Symbole	Signification
$P \in \mathbb{R}^3$	Point de l'espace, de coordonnées (x, y, z).
mathbfp	Vecteur de position, à trois coordonnées.
$R \in SO(3)$	Rotation, élément du groupe spécial orthogonal.
$T \in SE(3)$	Rigidification d'une rotation et d'une translation.
Π	Projection perspective d'un point de l'espace sur le plan image.
K	Matrice intrinsèque d'une caméra.
T_{ext}	Matrice extrinsèque, qui lie la caméra au repère du monde.
X_c	Coordonnées d'un point dans le repère de la caméra.
u, v	Coordonnées du pixel dans l'image.
M = (V, F)	Maillage : sommets et faces.
|V|	Nombre de sommets du maillage.
|F|	Nombre de faces du maillage.
d	Distance métrique entre deux points, ou entre deux images.
σ	Écart-type d'une grandeur.
χ^2	Distance du chi-deux, pour l'ajustement d'un modèle.

Tableau 1 – Symboles mathématiques et leur signification

> **Convention**
>
> Toute divergence constatée entre cette table et un chapitre signale une erreur de transcription, non une définition nouvelle : la table fait foi.

> **A retenir**
>
> - Un sigle employé sans être listé ici est un oubli de l'auteur, non une définition nouvelle.
> - Un symbole qui change de sens d'un chapitre à l'autre rend toute démonstration ininterprétable : c'est la principale cause des contresens que nous avons relevés.
> - Ces deux tables sont les seules parties du cours à maintenir à la main : le reste se relit tout seul.

Index des figures et des tableaux

Référence rapide du matériel visuel

Figures

1. Les modèles de génération 3D et ce que chacun optimise
1. De l'image au maillage exploitable : la chaîne complète
1. Quatre familles d'infrastructures, quatre compromis
1. Cinématique inverse : de la position cible aux angles
1. De la vidéo à la pose : les trois états du problème
1. Trois sources d'écart entre le simulateur et le réel
2. Physique différentiable : deux prédictions, une somme

Tableaux

1. Les familles de modèles et leurs compromis

1. Les quatre familles en détail
1. Les trois familles de résolution et leurs propriétés
1. Trois états du problème, trois contrats
1. Trois sources d'écart entre le simulateur et le réel
1. Abréviations employées dans le cours
1. Symboles mathématiques et leur signification

Fin du document

Intelligence Artificielle Avancée & Federated Learning — Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.
Ghiloufi Jamel, Architecte Logiciel Principal — Pixel Software Design
Département de Recherche & Développement — Recherche
Pixel Software Design — Édition 2026

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Document généré automatiquement le 29/09/2026 – mise en page Pixel Software Design.